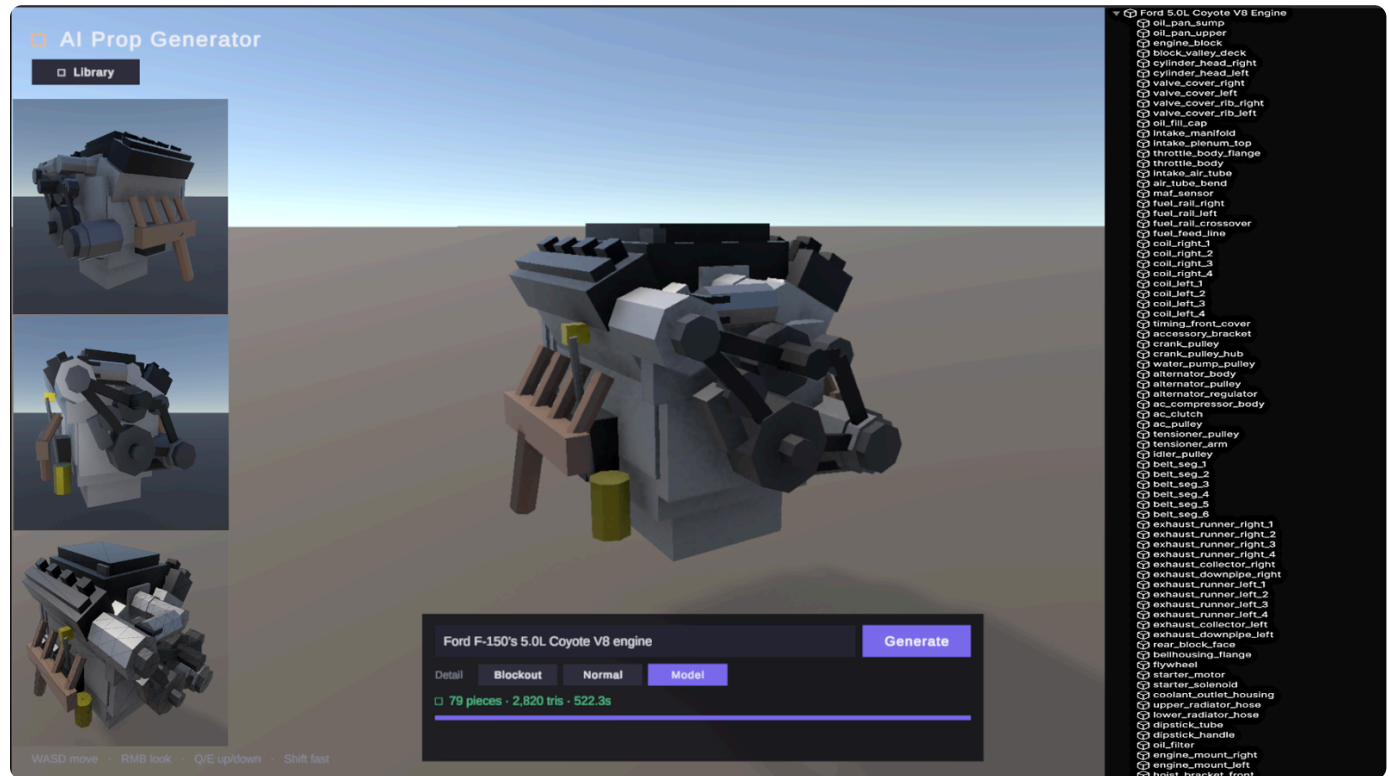


AI Runtime 3D Generator

Michael Dallara — Creator & Developer

An LLM-driven system, built in Unity, that turns natural-language descriptions into real, structured 3D models — generated at runtime, during live gameplay, with no pre-made assets. The pipeline is **model-agnostic**: it runs on any LLM capable of structured output, with the active model switchable live, in-game.

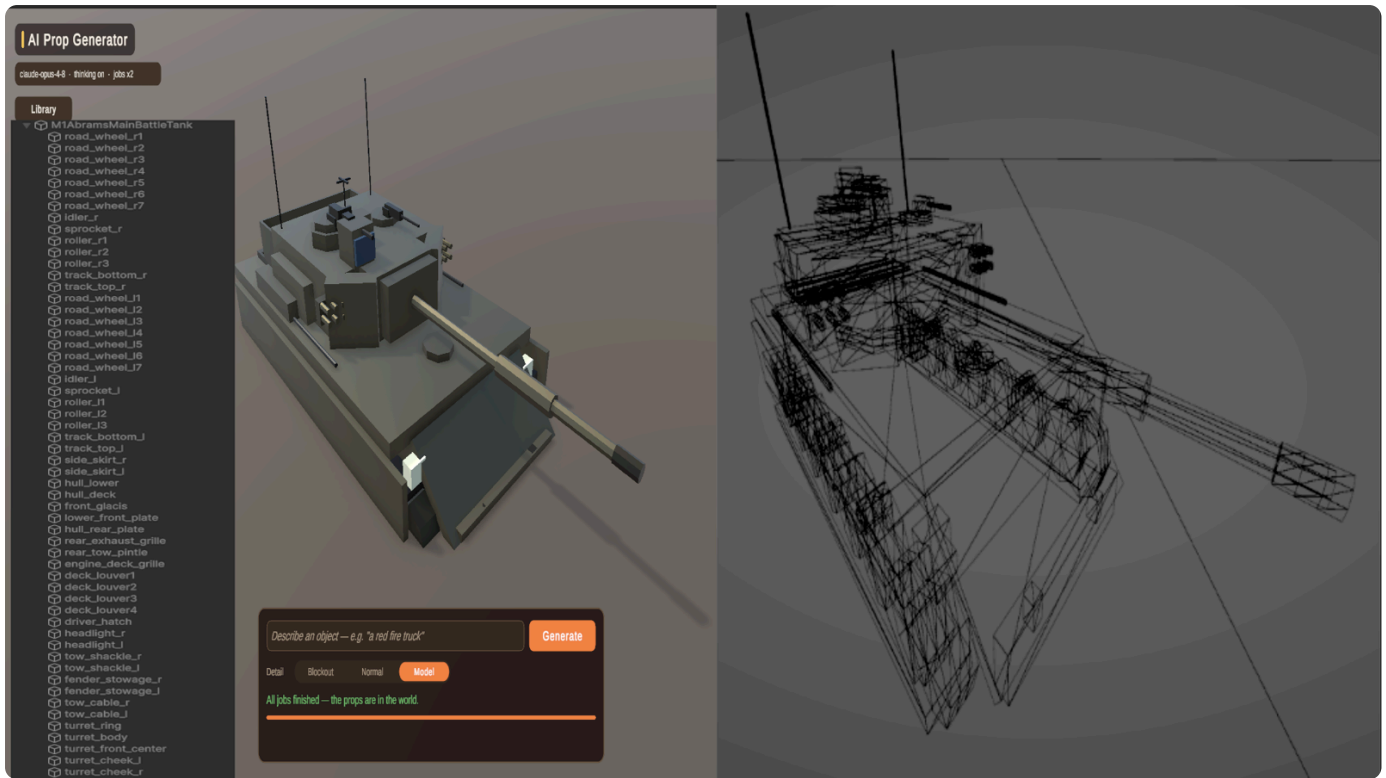
Solo personal project · 4–5 months of iterative R&D, ongoing · Sole designer, architect & developer



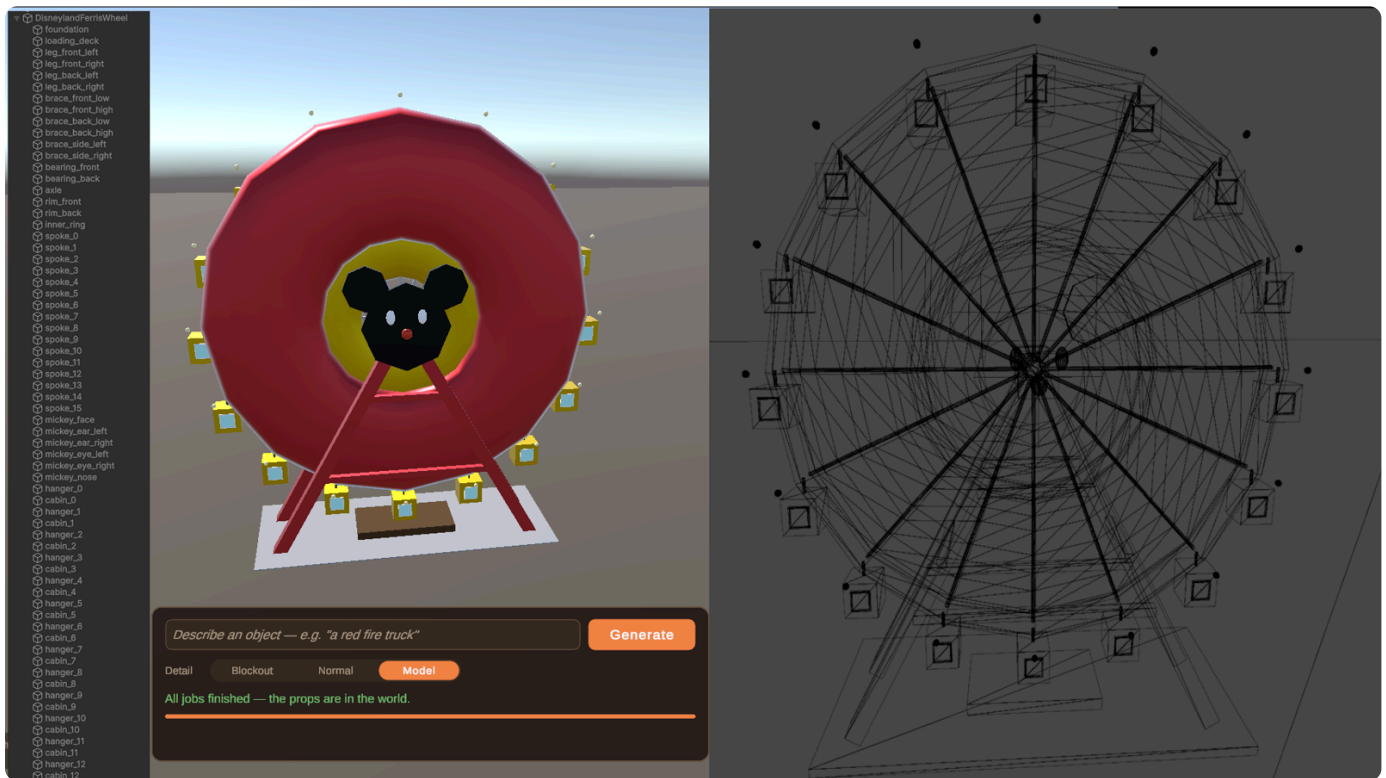
A Ford 5.0L Coyote V8 generated from one sentence — 79 named parts, 2,820 triangles, built live in-game at Model fidelity. Left: library thumbnails from other runs; right: the actual output part tree.

What it does

- You type a description — "a Ford EcoBoost V6," "an aircraft carrier," "a windmill for a farm" — and it constructs a real 3D model live in the running game, not in an offline editor step.
- **Selectable fidelity per generation** — a Blockout / Normal / Model dial sets how far the system goes for the same prompt: a 15-shape greybox that conveys the idea, a standard game-ready build, or a maximum-detail near-replica (an engine with individual cylinder banks, valve covers, intake runners, pulleys and belts as separate parts).
- **Concurrent, fire-and-forget generation** — submit a request and keep working: each job claims a spot in the world with a placeholder block and a floating status billboard (live progress, always facing the camera), runs in parallel with other jobs, and the finished model materializes in place. Placeholders can be dragged elsewhere while their job generates.
- Enables dynamic, player-driven content: objects generated on the fly from player input instead of being authored ahead of time.
- Produces recognizable results across wildly different domains — mechanical parts, engines, vehicles, buildings, ships, and infrastructure.



"M1 Abrams main battle tank" — road wheels, idlers, sprockets, side skirts, hatches and deck fittings as individually named, selectable parts. Right: the same model in wireframe — real structured geometry, not a mesh shell.

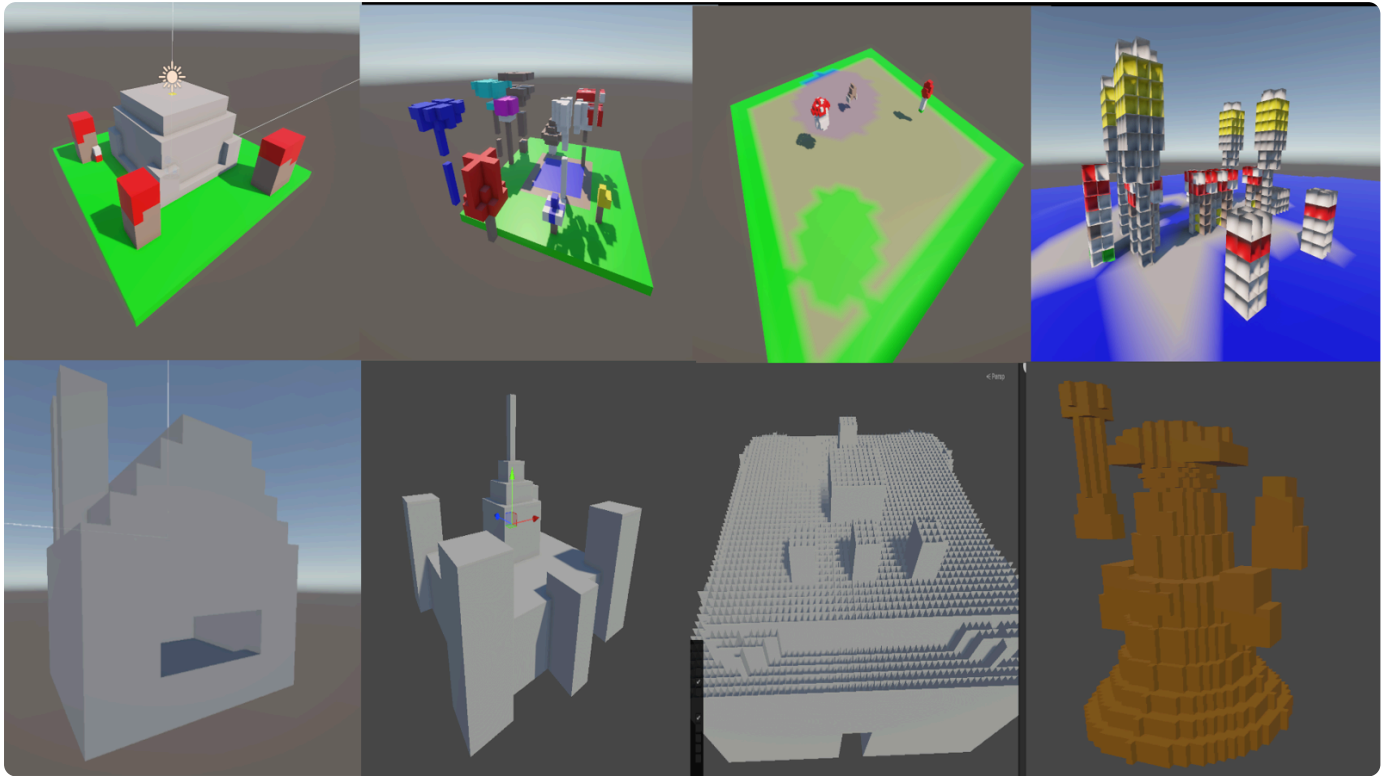


"Disneyland Ferris wheel" — the system inferred the Mickey-face hub on its own. Sixteen spokes with paired hangers and cabins placed by circular array, over a foundation and loading deck.

Design evolution

The current architecture is the survivor of a longer research arc, not a first attempt. Across months of prototyping I evaluated fundamentally different generation strategies — voxel-based construction among other geometry representations — and re-tested the approach as each new model generation raised the ceiling on what the

probabilistic side could do. The constrained-blueprint design won on the criteria that matter for a real system: reproducibility, editability of the output, and a deterministic core that can be unit-tested. Treating model capability as a moving target also shaped the engineering — the pipeline is deliberately model-agnostic, so every improvement in reasoning quality translates directly into better builds with zero code changes.



Where it started: voxel-based generation. Recognizable volumes — castles, terrain, a chess piece — but monolithic and unlabeled: nothing to select, name, or animate. This is what drove the pivot to part-level blueprints.

Architecture & how it works

- **Probabilistic / deterministic separation by design.** The LLM never generates geometry. It emits a compact, constrained JSON "blueprint" — a sequence of operations (spawn / repeat / array) over primitive forms, each with a bounding-box size and a position. A hand-written deterministic C# engine executes that blueprint into real meshes at runtime.
- **LLM-agnostic by design — and in practice.** The model's only responsibility is emitting the blueprint schema; everything vendor-specific lives behind one thin client interface. Any LLM capable of structured output can drive the pipeline: the build ships with Anthropic's Claude family plus an OpenAI-compatible client covering most of the ecosystem (OpenAI, Groq, Mistral, Gemini — or a free local model via Ollama), so plugging in a new provider is a config entry (base URL + model id + API key), hot-swappable from the in-game settings panel.
- **Two-pass agentic pipeline.** Pass one builds structure (geometry); pass two assigns materials and color (paint) — two constrained LLM calls bracketing a deterministic core. Default configuration: Claude Opus 4.8 with adaptive extended thinking.
- **Fidelity as a first-class system.** Each detail level is a tunable preset bundling the things that must move together — prompt directive, shape budget, token budget, reasoning effort, timeout, and paint strategy. The shape budget is a single number enforced at both ends: stated to the model in the prompt and applied as the engine's hard limit, so the instruction and the enforcement can never disagree.
- **Calibrated spatial contract.** Every shape obeys one unified rule (size = bounding box, position = center), verified against a reference "calibration sheet," keeping the model's output and the engine's interpretation in exact lockstep.

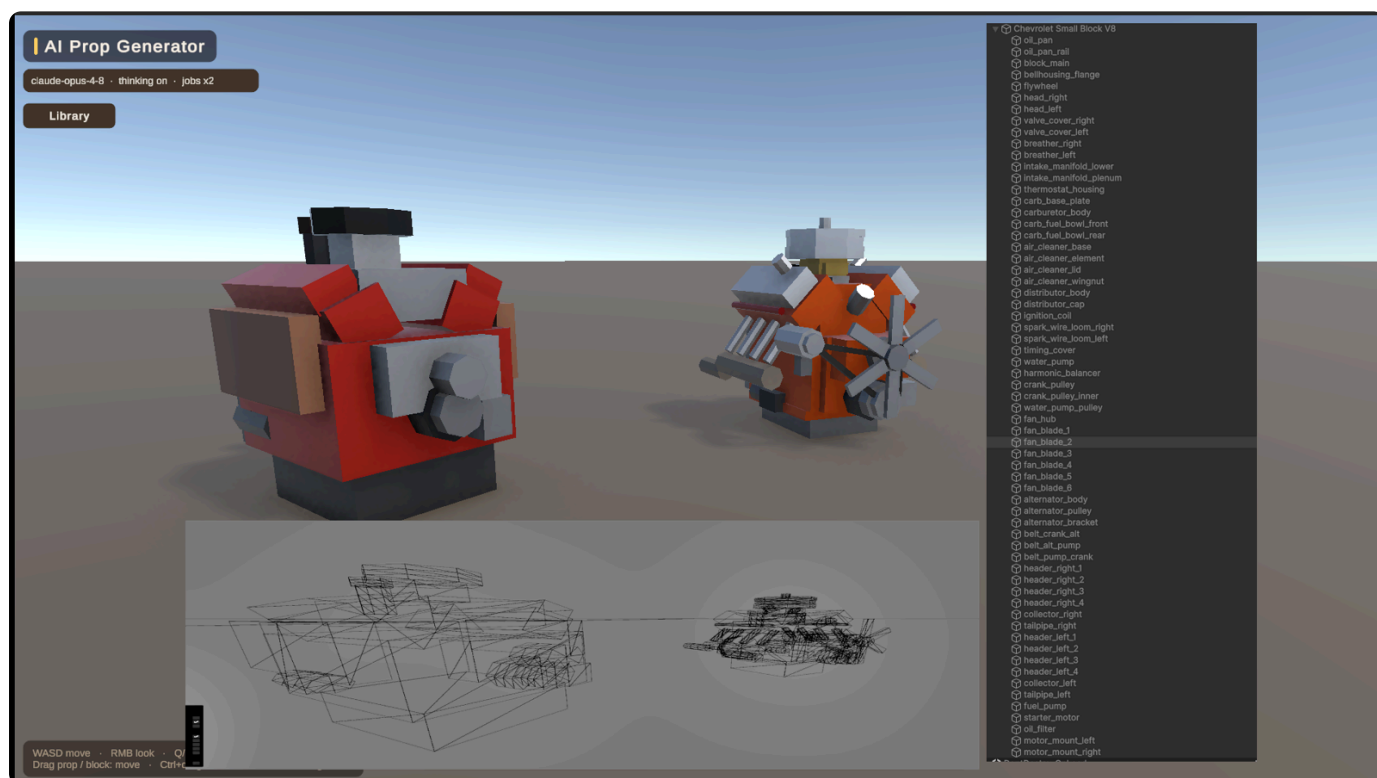
- **Zero external assets.** All geometry is generated at runtime from a small primitive set plus a self-made material palette — and the entire UI (panels, rounded-corner sprites, world-space status billboards) is built in code at startup, down to procedurally generated 9-sliced sprites. No model libraries, asset packs, or imported content of any kind.
- **Robustness.** Input/schema validation (rejects degenerate sizes, detects circular parent references), resilient JSON extraction from model output, deterministic processing, crash-safe persistence (atomic index writes with corrupt-file recovery), leak-free material/asset lifecycle, and clean editor/player build separation.

Detailed hierarchical decomposition

Every generation produces a single root object containing the model's entire part tree, broken down the way the real object is:

- Each part is its own named GameObject, labeled by function — e.g., an engine becomes `engine_block`, `cylinder_bank_left/right`, `valve_cover_left/right`, `intake_manifold`, `turbo_left/right` with their downpipe / charge_pipe, `crank_pulley`, `alternator`, `serpentine_belt`, `coil_left_1/2/3`, and more.
- Composite elements are real sub-assemblies — a window is a frame + glass; a building's windows and doors are auto-grouped into container nodes (Windows, Doors).

Because every part is an individually addressable object, any single piece can be selected, moved, rotated, recolored, or deleted — and targeted in code (animate a piston, swing a door, swap a turbo, attach physics or gameplay logic to a named part). The output is a programmable, inspectable object graph, not a static mesh.



Two independent generations of a Chevy small-block V8 — same prompt, distinct valid designs. The hierarchy panel (right) is the live output: `oil pan`, `flywheel`, `per-blade fan`, `spark-wire looms`, `serpentine belts` — every entry an addressable GameObject. Below: the underlying wireframes.

Asset lifecycle & dual workflows

The key enabling property: every asset is simultaneously a real, editable Unity hierarchy and a portable JSON blueprint (pure data). That duality unlocks the full lifecycle:

- Save any generated asset to a searchable on-disk library (blueprint + metadata), with crash-safe atomic persistence.

- Re-spawn deterministically — the same blueprint always rebuilds the identical model, so saved assets are reproducible forever.
- Duplicate any asset instantly.
- Hand-edit — refine the JSON blueprint directly, or edit individual parts in-scene (move, raise/lower, rotate, recolor, remove pieces) with full mouse-driven manipulation.
- Refine & iterate — regenerate from a sharpened prompt, tweak by hand, then re-save the improved version.
- Build a permanent library — curate a growing set of AI-generated, human-refined assets you own and reuse like any other game content, indefinitely.

Workflow A — Author / library mode: generate, curate, fine-tune, and bank a reusable asset library for a project.

Workflow B — Runtime / in-game mode: generate content live, on demand, from player input while the game is being played. The same engine powers both — a content-creation pipeline and a live gameplay capability.

Key technical accomplishments

- Designed an LLM-to-structured-output architecture that makes generative results reproducible, verifiable, and unit-testable by constraining the model to a typed blueprint instead of free-form geometry.
- Made the pipeline **vendor-independent**: a provider-agnostic client interface with two wire-format implementations (Anthropic + OpenAI-compatible) covering hosted and local models, per-request budget overrides (tokens, reasoning effort, timeout), live in-app provider switching, and per-generation token & cost telemetry.
- Engineered a calibrated prompt ↔ engine contract that holds across a universal domain (a bolt to a battleship) — solving the core challenge of consistency in an open-ended builder.
- Built a **tiered fidelity system** — Blockout / Normal / Model presets that co-tune the prompt directive, shape cap, and API budgets, with the shape cap enforced identically on both sides of the AI boundary.
- Built a **concurrent job system** for parallel generation: a capped job queue, per-job state machines, and world-space status billboards over draggable placeholder blocks — fire-and-forget UX over long-running async AI calls in a single-threaded engine.
- Implemented a deterministic geometry engine (~14 primitive / composite shapes, bounding-box fitting, named hierarchical output) with EditMode unit tests on the core math.
- Built a full data-driven asset lifecycle (save, deterministic re-spawn, duplicate, hand-edit, refine into a persistent library) and a fully code-built, self-wiring runtime UI — themed panels, segmented controls, and an in-game settings hub, rendered with procedurally generated sprites and zero art assets.
- Production-hardened the codebase: atomic on-disk persistence with corrupt-file recovery, leak-free material/asset lifecycle, editor/player build separation, and gated diagnostics.

Tech stack

Languages / Engine: C#, Unity 6.3. **AI:** LLM-agnostic pipeline — Anthropic Claude API by default (Opus 4.8 with adaptive extended thinking), with any OpenAI-compatible endpoint or local model (Ollama) pluggable via config and switchable in-app. **3D / Rendering:** ProBuilder (runtime procedural mesh generation), Universal Render Pipeline, TextMeshPro. **Data / Tooling:** Newtonsoft JSON, Unity Test Framework (EditMode), Git.

Skills demonstrated

LLM application development · AI engineering · comparative prototyping / architecture evaluation · prompt engineering · structured / constrained generation · JSON schema · agentic / multi-step LLM pipelines · model-agnostic LLM integration · Anthropic Claude API · extended / adaptive thinking · deterministic systems design · reproducibility · async & concurrent systems · software architecture · separation of concerns · C# · Unity ·

Scope & status

- Single descriptions yield models of dozens of organized parts (e.g., a generated V6 engine $\approx$ 30+ labeled components); Model-fidelity builds scale to a 200-shape budget.
- Demonstrated range: a single bolt $\rightarrow$ a labeled engine $\rightarrow$ buildings $\rightarrow$ a full farm scene $\rightarrow$ an aircraft carrier, a naval fleet, and a SpaceX-style launch complex.
- **Status:** polished, stability-hardened beta. Geometry is blockout / prototype quality (clean primitive-based meshes); generation takes seconds to a couple of minutes depending on fidelity; solo-built. Best framed as a strong working system demonstrating AI-systems architecture and engineering judgment.